AI 代驾数据集接入 VLMEvalKit 开发文档

概述

本文档介绍如何将 AI Designated Driver（AI 代驾）自定义数据集接入到 VLMEvalKit 评测框架中。该数据集包含三个子任务：

- **异常行为检测** (abnormal_detection)
- **身份识别/马甲检测** (clothes_detection)
- **失物检测** (lost_items_detection)

数据集格式

数据文件结构

数据集采用 JSON 格式，遵循 ShareGPT 格式规范：

代码块

```
[
  {
    "messages": [
      {
        "role": "system",
        "content": "系统提示词，包含任务规范和输出格式要求"
      },
      {
        "role": "user",
        "content": "<image><image>...请根据这些图像判断..."
      },
      {
        "role": "assistant",
        "content": "[]"  // 或 "[动作@位置]" 等格式
      }
    ],
    "images": [
      "/path/to/image1.jpg",
      "/path/to/image2.jpg"
    ],
    "label": "0",  // 或 "1"
    "task": "abnormal_detection"  // 任务类型
  }
]
```

数据集文件位置

数据集文件应放置在以下任一位置（按优先级顺序）：

1. `$LMUData/ai_designated_driver_datav5_2/`
2. 项目根目录下的 `ai_designated_driver_datav5_2/`
3. 当前工作目录下的 `ai_designated_driver_datav5_2/`

支持的数据集

- `AIDesignatedDriver_eval` - 综合评估数据集
- `AIDesignatedDriver_eval_abnormal_detection` - 异常行为检测
- `AIDesignatedDriver_eval_clothes_detection` - 身份识别/马甲检测
- `AIDesignatedDriver_eval_lost_items_detection` - 失物检测

实现细节

1. 数据集类实现

文件位置: `vlmeval/dataset/ai_designated_driver.py`

核心类: `AIDesignatedDriverDataset`

该类继承自 `ImageVQADataset`，实现了以下关键方法：

1.1 `load_data()` 方法

从 JSON 文件加载数据并转换为 pandas DataFrame：

- 从 `messages` 中提取 `system`、`user`、`assistant` 消息
- 从 `user` 消息中提取问题（移除 `<image>` 标签）
- 从 `assistant` 消息或 `label` 字段提取答案
- 处理图像路径（支持单图和多图）
- 保存 `system_prompt` 用于后续构建提示词

关键特性：

- 支持多图像输入
- 自动查找数据文件（支持多个可能位置）
- 错误处理和警告提示

1.2 `build_prompt()` 方法

构建多模态消息列表，格式符合 VLMEvalKit 要求：

代码块

```
[
    dict(type='text', value=system_prompt, role='system'),  # system 消息
    dict(type='image', value=image_path1, role='user'),     # 图像1
    dict(type='image', value=image_path2, role='user'),     # 图像2（如果有）
    dict(type='text', value=question, role='user')          # 问题
]
```

关键特性：

- 包含 system 提示词（重要：确保模型遵循输出格式）
- 支持多图像输入
- 智能处理图像路径（绝对路径、相对路径、文件查找）

1.3 `evaluate()` 方法

评估预测结果，支持自定义评估逻辑：

评估方法：

- 使用精确匹配（忽略大小写和首尾空格）
- 特殊处理 `'[]'` 格式的答案（作为字符串而非空列表）
- 支持按任务类型分组统计
- 计算总体准确率

关键实现：

- 自定义 `process_line_custom()` 函数处理答案格式
- 使用 multiprocessing 并行处理
- 避免 `nan` 值（空列表时返回 0.0）

2. 评估函数实现

模块级函数: `process_line_custom()`

该函数必须定义在模块级别（而非类方法内部），以便被 multiprocessing 序列化。

功能：

- 处理 `'[]'` 格式的答案（作为字符串 `'[]'` 而非空列表）
- 处理 `'[代驾马甲]'` 等格式的答案

- 精确匹配（忽略大小写和首尾空格）

关键逻辑：

```
代码块
if answer == '[]':
    answers = ['[]']  # 作为字符串处理
elif answer.startswith('[') and answer.endswith(']'):
    # 尝试解析，但空列表仍作为 '[]' 字符串处理
    ...
```

3. 注册到框架

文件位置: `vlmeval/dataset/__init__.py`

3.1 导入数据集类

在 `__init__.py` 中添加导入：

```
代码块
from .ai_designated_driver import AIDesignatedDriverDataset
```

3.2 添加到数据集列表

将数据集类添加到 `IMAGE_DATASET` 列表：

```
代码块
IMAGE_DATASET = [
    ...
    AIDesignatedDriverDataset,  # 添加这一行
]
```

使用方法

1. 准备数据文件

将数据集 JSON 文件放置在以下位置之一：

- `$LMUData/ai_designated_driver_datav5_2/`
- 项目根目录下的 `ai_designated_driver_datav5_2/`

确保图像文件路径正确（可以是绝对路径或相对于数据集目录的相对路径）。

2. 运行评测

使用 `run.py` 脚本运行评测：

代码块

```
# 评测单个数据集
python run.py --data AIDesignatedDriver_eval_clothes_detection --model vllm

# 评测多个数据集
python run.py --data AIDesignatedDriver_eval_abnormal_detection AIDesignatedDriver_eval_clothes_detection --model vllm

# 仅评估已有结果
python run.py --data AIDesignatedDriver_eval_clothes_detection --model vllm --eval-only
```

3. 查看结果

评测结果保存在 `outputs/<model_name>/` 目录下：

- `*_results.xlsx` - 详细评估结果
- `*_acc.csv` - 准确率统计（按任务分组）

关键技术点

1. System 提示词处理

问题：原始实现未包含 system 提示词，导致模型输出格式不符合要求。

解决方案：

- 在 `load_data()` 中提取并保存 `system_prompt`
- 在 `build_prompt()` 中将 system 消息添加到消息列表开头
- 消息格式包含 `role='system'` 字段，API 会自动处理

2. 答案格式处理

问题：`'[]'` 格式的答案被 `eval()` 解析为空列表，导致评估时 `match` 列表为空，产生 `nan`。

解决方案：

- 自定义 `process_line_custom()` 函数
- 特殊处理 `'[]'` 作为字符串而非空列表
- 确保 `match` 列表始终包含至少一个元素

3. Multiprocessing 序列化

问题：局部函数无法被 multiprocessing 序列化。

解决方案：

- 将 `process_line_custom()` 定义为模块级函数
- 确保函数可以被 pickle 序列化

4. 图像路径处理

问题：数据中的图像路径可能是绝对路径、相对路径，或路径不存在。

解决方案：

- 智能路径查找：优先使用存在的绝对路径
- 多位置查找：数据集目录、项目目录等
- 容错处理：路径不存在时仍返回原始路径（由后续处理决定）

文件结构

代码块

```
VLMEvalKit/
├── vlmeval/
│   └── dataset/
│       ├── __init__.py                    # 注册数据集类
│       ├── ai_designated_driver.py        # 数据集类实现
│       └── ...
├── ai_designated_driver_datav5_2/         # 数据集目录
│   ├── qwen25vl_eval.json
│   ├── qwen25vl_eval_abnormal_detection.json
│   ├── qwen25vl_eval_clothes_detection.json
│   ├── qwen25vl_eval_lost_items_detection.json
│   └── ...
└── docs/
    └── AIDesignatedDriver_Dataset_Integration.md   # 本文档
```

注意事项

1. **图像路径**：确保图像文件存在，或路径可以正确解析
2. **数据格式**：JSON 文件必须符合 ShareGPT 格式
3. **System 提示词**：确保包含在 `messages` 中，用于规范模型输出
4. **答案格式**：答案格式必须与 system 提示词中规定的格式一致

5. **多进程安全**：评估函数必须是模块级函数，可被 pickle 序列化

扩展指南

添加新的子任务数据集

1. 在 `DATASET_FILES` 字典中添加新条目：

```
代码块
DATASET_FILES = {
    ...
    'AIDesignatedDriver_eval_new_task': 'qwen25vl_eval_new_task.json',
}
```

2. 确保 JSON 文件格式正确
3. 重新运行评测

修改评估方法

如需修改评估逻辑，编辑 `process_line_custom()` 函数：

```
代码块
def process_line_custom(line):
    # 自定义评估逻辑
    ...
    ret['match'] = [自定义匹配逻辑]
    return ret
```

支持其他数据格式

如需支持其他数据格式，可以：

1. 修改 `load_data()` 方法以支持新的数据格式
2. 或创建新的数据集类继承 `AIDesignatedDriverDataset`

总结

通过以上实现，AI Designated Driver 数据集已成功接入 VLMEvalKit 框架，支持：

- ✅ 多模态输入（文本 + 图像）
- ✅ System 提示词支持
- ✅ 自定义评估方法

- ✅ 多任务分组统计
- ✅ 与框架无缝集成

数据集可以像其他官方数据集一样使用，完全兼容 VLMEvalKit 的评测流程。